



Alliance School of Advanced Computing

Course Name: Graph Analysis

Course Code: E1ITC302

Assignment No: 1

Name of the Student: Tunga Naga Rupesh

Registration Number: 2411021060454

A. Title of The Problem:

Components in a Graph

B. Problem Statement in details:

There are $2 \times N$ nodes in an undirected graph, with edges connecting nodes from the first half $[1, N]$ to the second half $[N, +12N]$. The task is to determine the size of the smallest and largest connected components that contain at least 2 nodes. Single isolated nodes should not be considered.

C. HackerRank/Leetcode Link of the problem (If applicable):

[HackerRank – Components in a graph](#)

D. Student's Github link of report and code upload, along with the readme file:

[-GITHUB link](#)

E. Solution Steps/Algorithm:

- ☐ Build an adjacency list from the given edges.
- ☐ Use BFS to explore connected components.
- ☐ Track visited nodes to avoid repetition.
- ☐ For each component, calculate its size.
- ☐ Update smallest and largest component sizes (ignoring single nodes).

F. Code developed:

```
import java.util.*;

public class Solution {

    public static int[] connectedComponents(int[][] bg) {
        // Step 1: Build adjacency list
        Map<Integer, List<Integer>> graph = new HashMap<>();

        for (int[] edge : bg) {
            if (!graph.containsKey(edge[0])) {
                graph.put(edge[0], new ArrayList<Integer>());
            }
            if (!graph.containsKey(edge[1])) {
```

```
graph.put(edge[1], new ArrayList<Integer>());
}
graph.get(edge[0]).add(edge[1]);
graph.get(edge[1]).add(edge[0]);
}

// Step 2: Track visited nodes
Set<Integer> visited = new HashSet<>();
int smallest = Integer.MAX_VALUE;
int largest = Integer.MIN_VALUE;

// Step 3: Explore components
for (int node : graph.keySet()) {
    if (!visited.contains(node)) {
        int size = bfs(node, graph, visited);
        if (size > 1) { // ignore single nodes
            smallest = Math.min(smallest, size);
            largest = Math.max(largest, size);
        }
    }
}

return new int[] {smallest, largest};
}

// BFS helper
private static int bfs(int start, Map<Integer, List<Integer>> graph, Set<Integer> visited) {
    Queue<Integer> q = new LinkedList<>();
    q.add(start);
```

```
visited.add(start);

int count = 0;

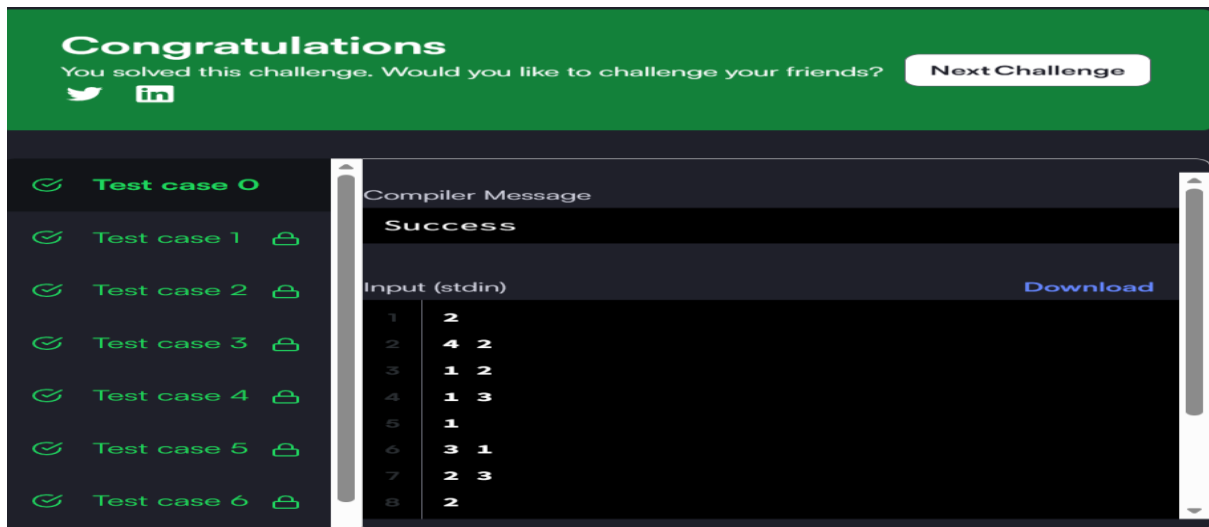
while (!q.isEmpty()) {
    int node = q.poll();
    count++;
    for (int neighbor : graph.get(node)) {
        if (!visited.contains(neighbor)) {
            visited.add(neighbor);
            q.add(neighbor);
        }
    }
}

return count;
}

// Main method for testing
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt(); // number of edges
    int[][] bg = new int[n][2];
    for (int i = 0; i < n; i++) {
        bg[i][0] = sc.nextInt();
        bg[i][1] = sc.nextInt();
    }

    int[] result = connectedComponents(bg);
    System.out.println(result[0] + " " + result[1]);
}
```

G. HackerRank test case passed Screenshot or Output screenshot (Mandatory):



H. Observation:

The BFS traversal correctly identifies connected components. The algorithm efficiently computes the smallest and largest component sizes, excluding isolated nodes.